The DARK SIDE of Microservices

Pitfalls and Antipatterns

Steve “ardalis” Smith
@ardalis.com | ardalis
steve@nimblepros.com | NimblePros.com

NimblePros
Better software development. Delivered.

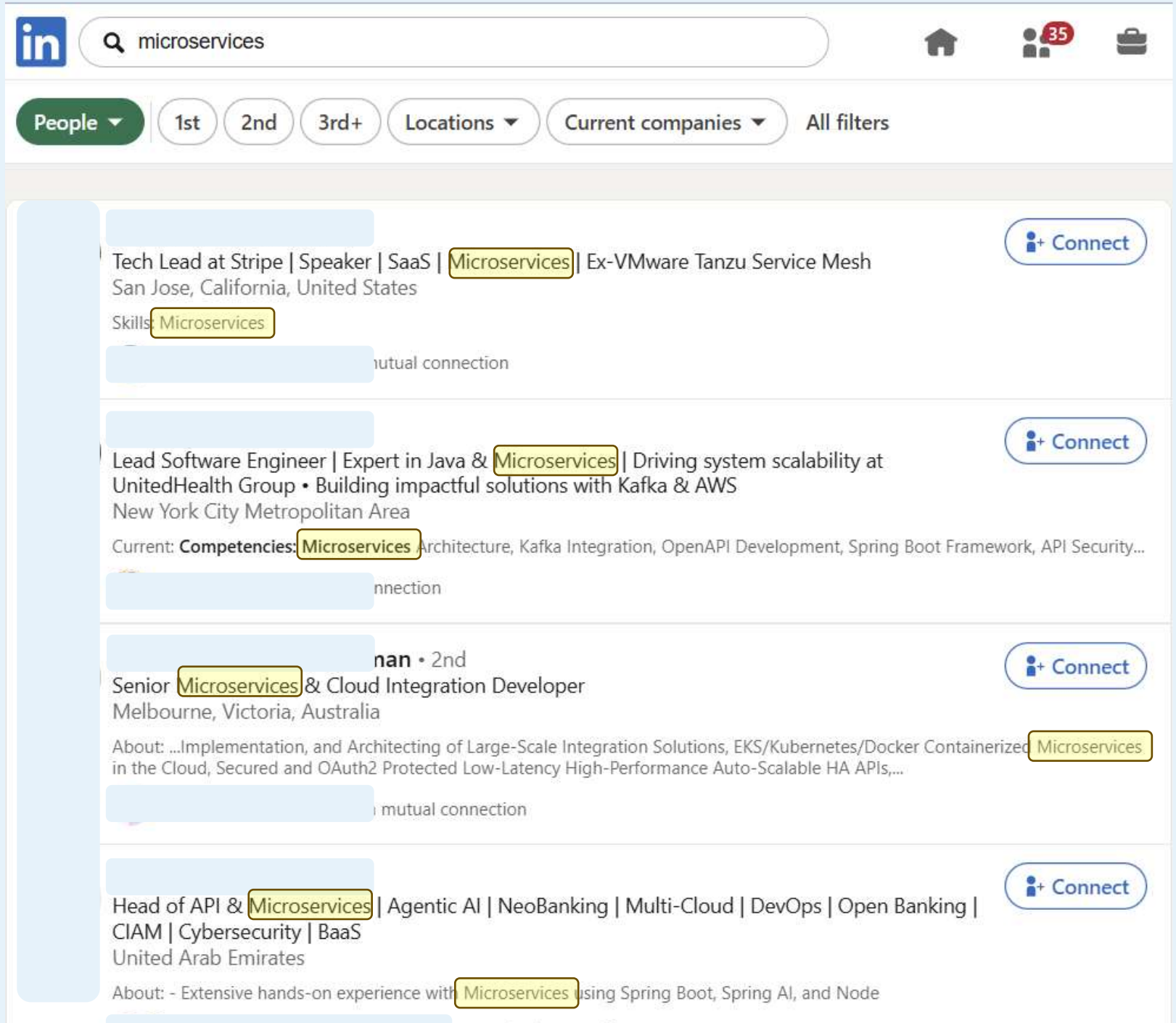
The Last Decade…

Why?

Mostly? LIO

LinkedIn® Optimization

LinkedIn:


in
microservices
35
People
1st
2nd
3rd+
Locations
Current companies
All filters
Connect
Tech Lead at Stripe | Speaker | SaaS | Microservices | Ex-VMware Tanzu Service Mesh
San Jose, California, United States
Skills: Microservices
utual connection
Connect
Lead Software Engineer | Expert in Java & Microservices | Driving system scalability at UnitedHealth Group • Building impactful solutions with Kafka & AWS
New York City Metropolitan Area
Current: Competencies: Microservices Architecture, Kafka Integration, OpenAPI Development, Spring Boot Framework, API Security...
nnection
Connect
nan • 2nd
Senior Microservices & Cloud Integration Developer
Melbourne, Victoria, Australia
About: ...Implementation, and Architecting of Large-Scale Integration Solutions, EKS/Kubernetes/Docker Containerized Microservices in the Cloud, Secured and OAuth2 Protected Low-Latency High-Performance Auto-Scalable HA APIs,...
mutual connection
Connect
Head of API & Microservices | Agentic AI | NeoBanking | Multi-Cloud | DevOps | Open Banking | CIAM | Cybersecurity | BaaS
United Arab Emirates
About: - Extensive hands-on experience with Microservices using Spring Boot, Spring AI, and Node

FOMO

Fear Of Missing Out

everybody else is doing it so why can't we?

By 2016

All of these companies were touting their moves to microservices

amazon

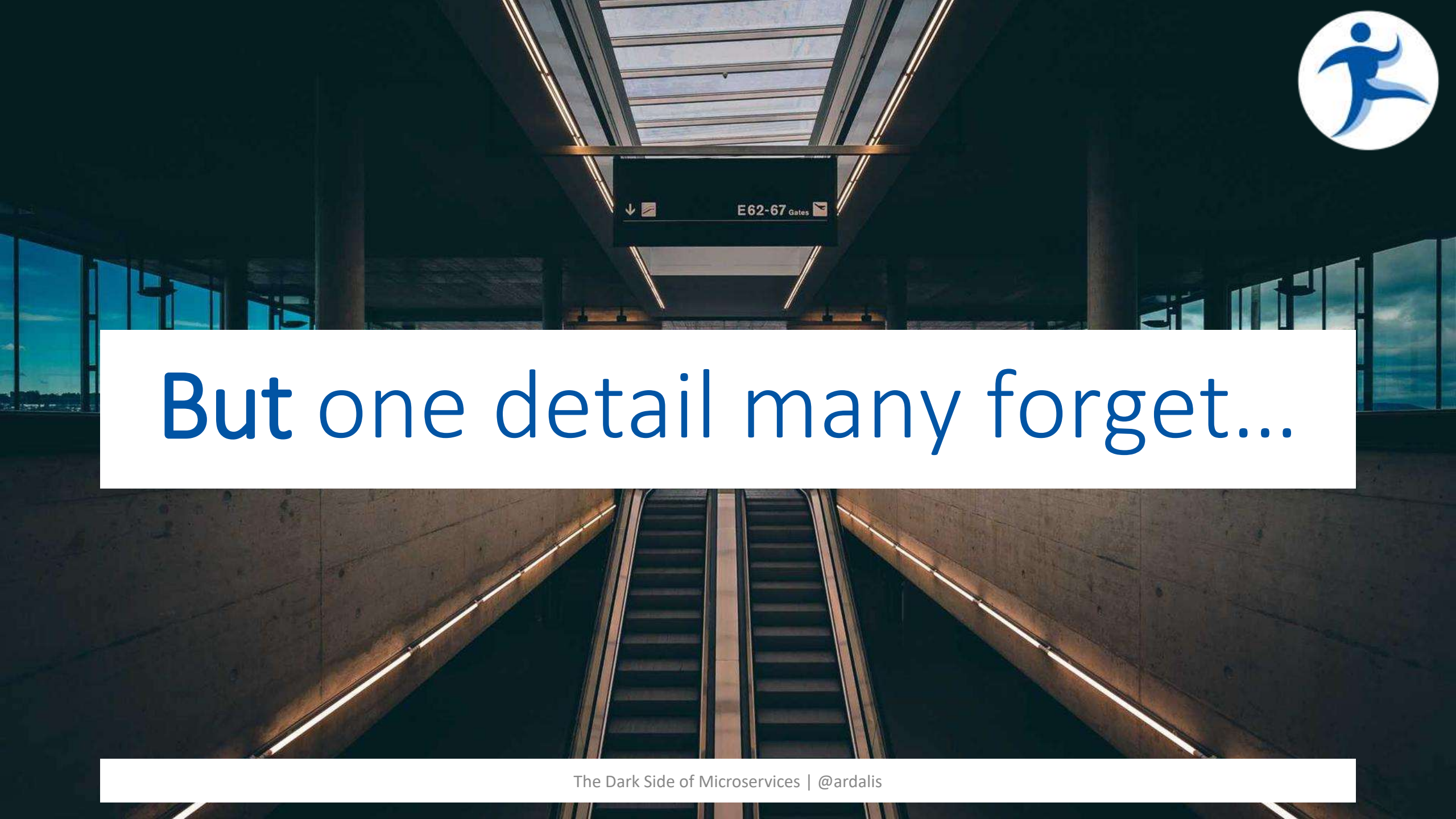
BEST BUY

ebay

NETFLIX

Etsy

Coca-Cola



But one detail many forget…

None of these companies **started** with microservices!

Microservice Envy

Beware of chasing technology patterns based on other companies' accounts

Of course other companies are going to talk about how great things are going for them with their technology choices…

Even if it's true:
Microservices work great at %COMPANY%

That doesn't mean they're the right choice for your app! **It Depends™** on context!

- Team Size
- Infrastructure Budget
- App Complexity
- *All* Microservices or just *Some* Microservices
- Team Experience
- Time to Market Pressure
- Engineering Org Size / Experience

Ideal Microservices

The API Mandate, Jeff Bezos email to all Amazon employees, 2002

2002
To: All Amazon Employees

1. *All teams will henceforth expose their data and functionality through service interfaces.*
2. *Teams must communicate with each other through these interfaces.*
3. *There will be no other form of interprocess communication allowed: no direct linking, no direct reads of another team's data store, no shared-memory model, no back-doors whatsoever. The only communication allowed is via service interface calls over the network.*
4. *It doesn't matter what technology they use. HTTP, CORBA, Pubsub, custom protocols — doesn't matter.*
5. *All service interfaces, without exception, must be designed from the ground up to be externalizable. That is to say, the team must plan and design to be able to expose the interface to developers in the outside world. No exceptions.*
6. *Anyone who doesn't do this will be fired.*
7. *Thank you; have a nice day!*

https://chrislaing.net/blog/the-memo/

2026 AWS Docs Microservices Characteristics

Autonomous

Autonomously
- Developed
- Deployed
- Operated
- Scaled

…without affecting other services

Communicate via well-defined APIs

Specialized

A set of capabilities focused on solving a specific problem

If it becomes too complex, it can be broken into smaller services

https://aws.amazon.com/microservices/

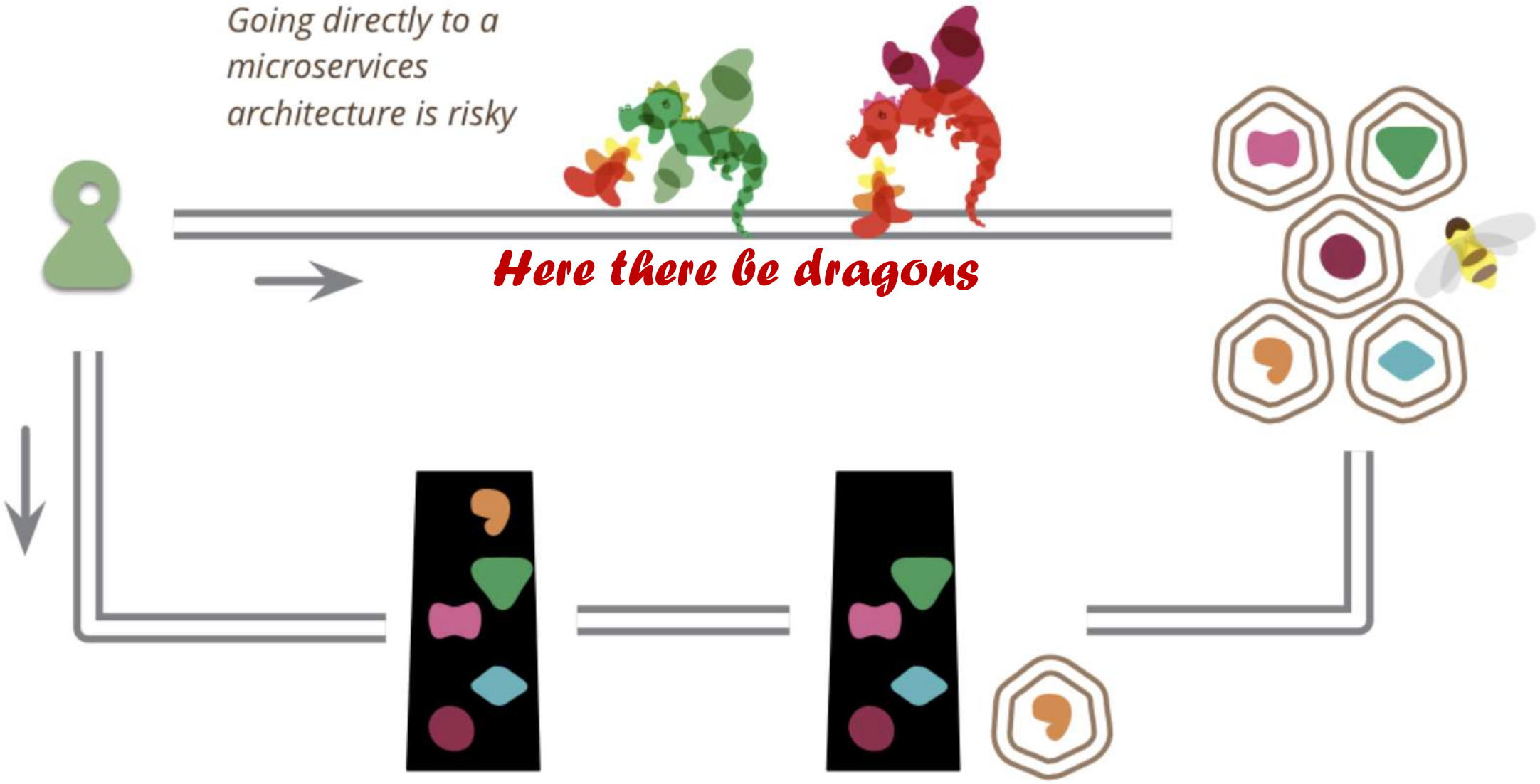
Microservice Pitfalls

Pitfall 1
Breaking The First Rule of Distributed Systems

Don’t distribute your system unless you have an observable reason to.

"We do these things not because they are easy,
but because we thought they would be easy."

Microservice Engineers, probably



https://martinfowler.com/bliki/MonolithFirst.html

Starting with a (modular) monolith and then pulling out individual services is less risky than a big bang rewrite (or v1 microservices app).

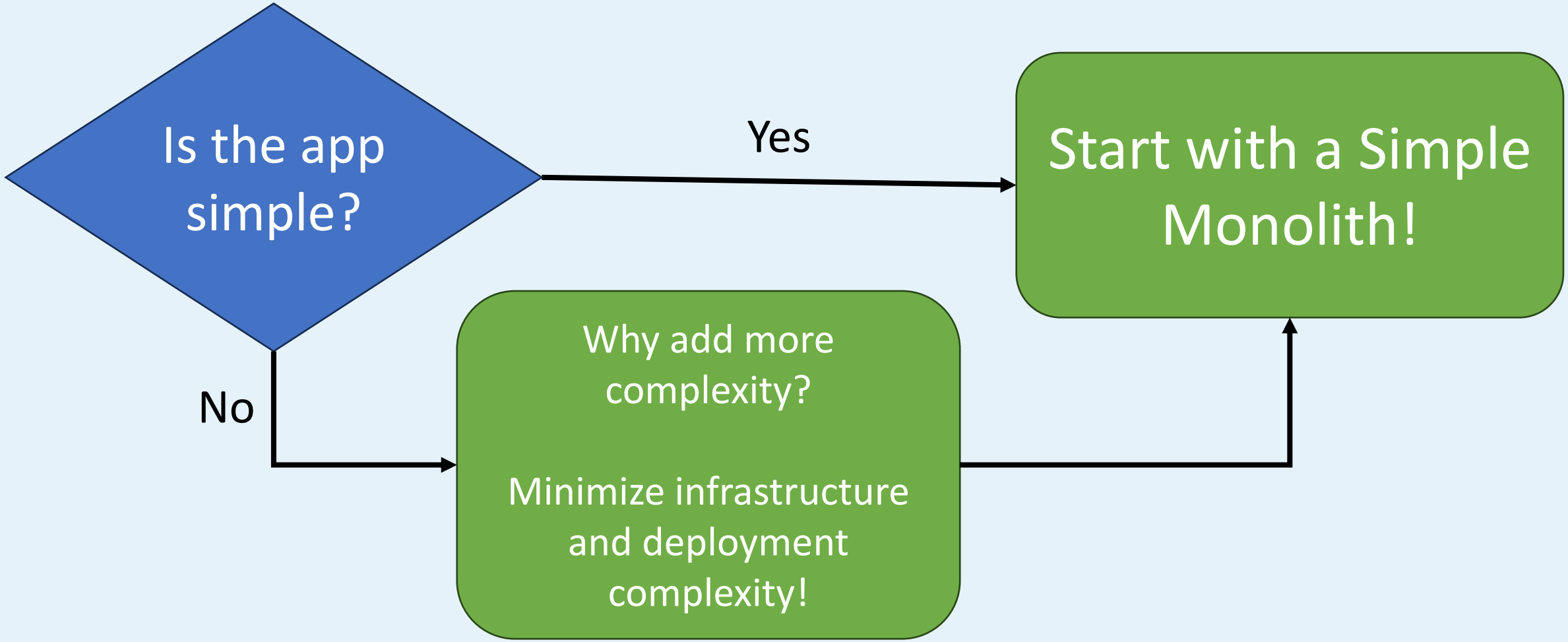
Fallacies of Distributed Systems

- The network is reliable
- Latency is zero
- Bandwidth is infinite
- The network is secure
- Topology doesn’t change
- Etc

All of these come into play with the introduction of microservices

https://en.wikipedia.org/wiki/Fallacies_of_distributed_computing

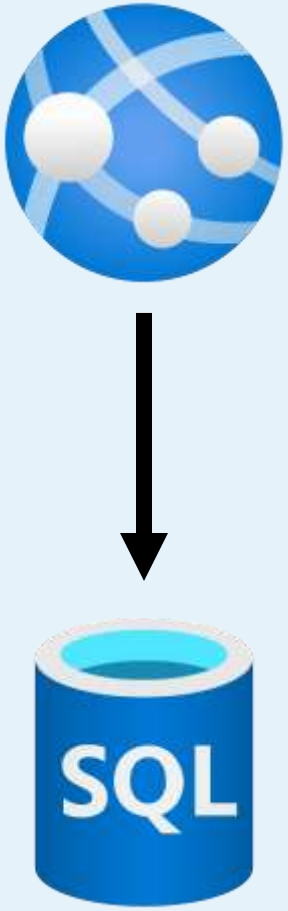
Easy New App Microservices Decision Matrix



Pitfall 2
Sharing the Database

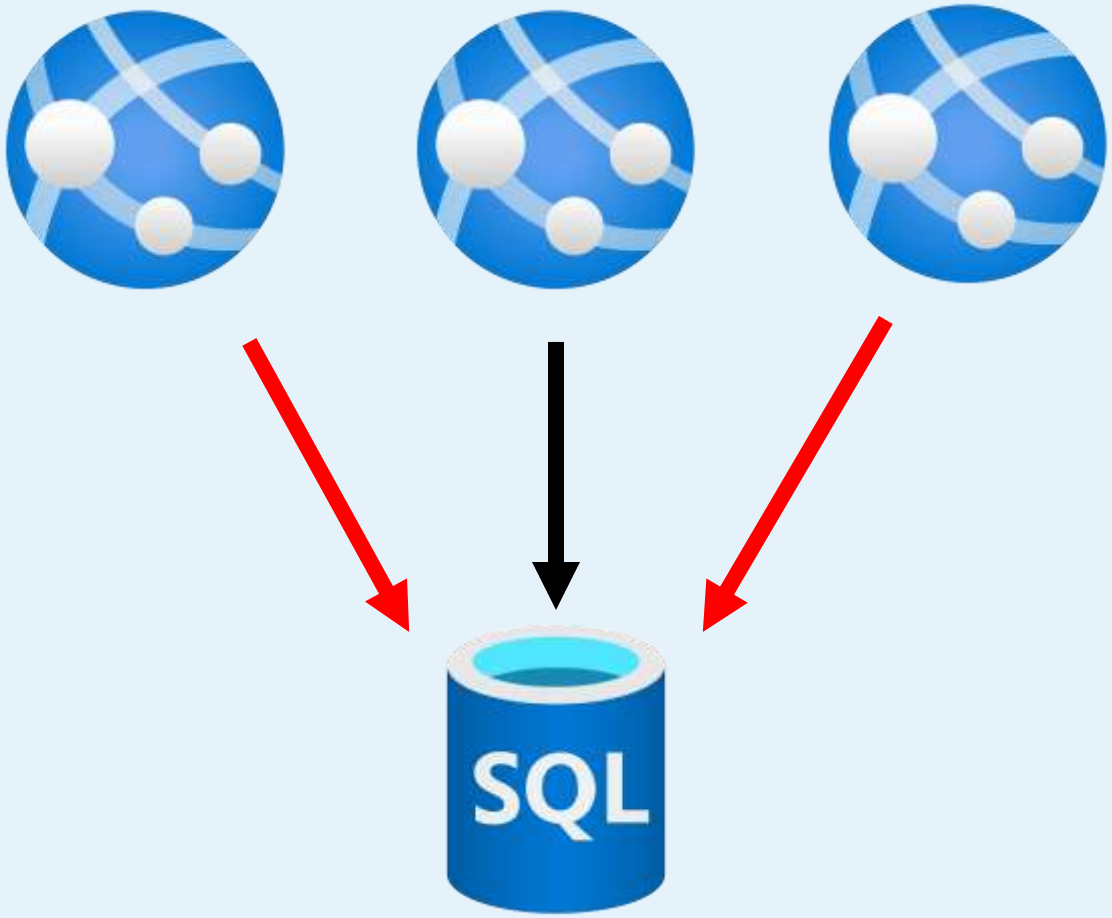
"Let's split up the logic, but keep the data where it is for now…"

A Single App



One team owns the database and application

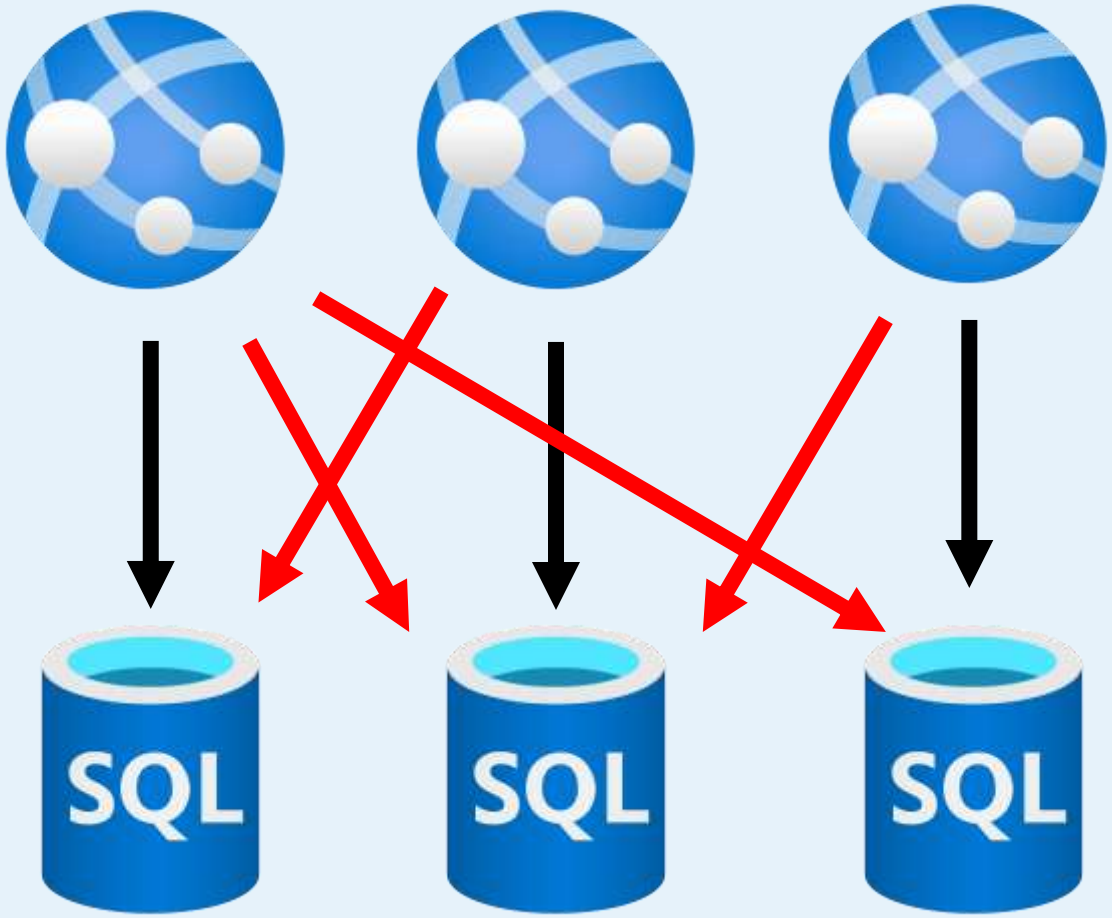
Several Apps/Services



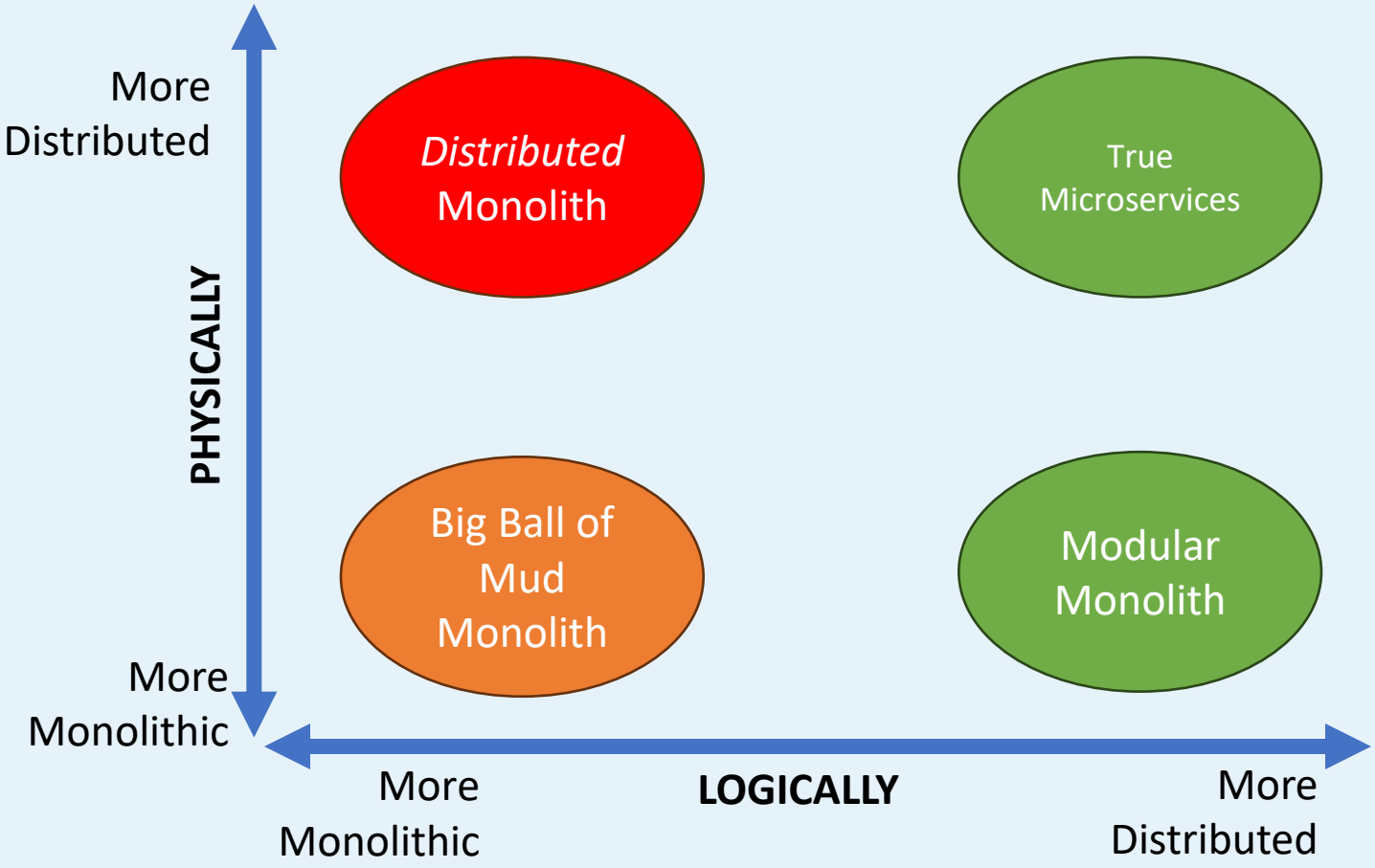
One App's DB Updates can bring down other apps

Database per App!

The Distributed Monolith

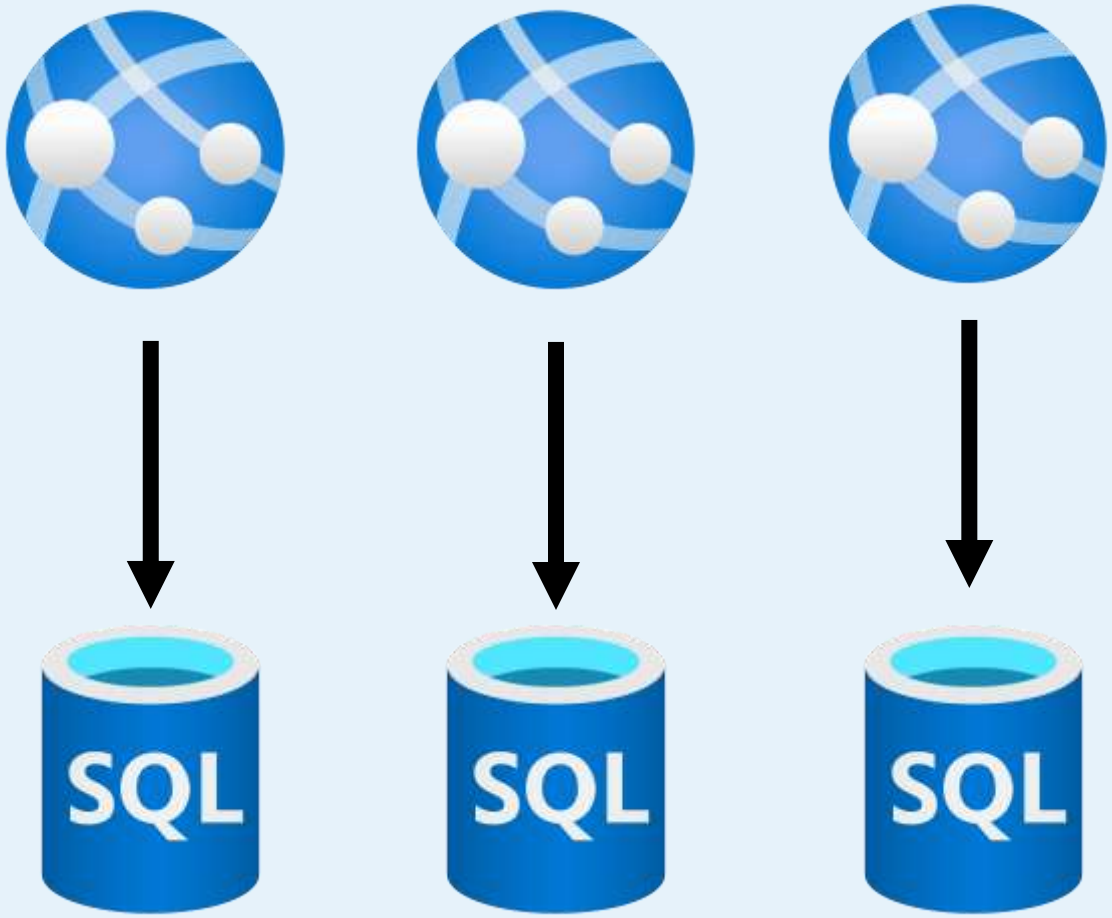


Good and Bad Monoliths



https://blog.ttulka.com/good-and-bad-monolith/

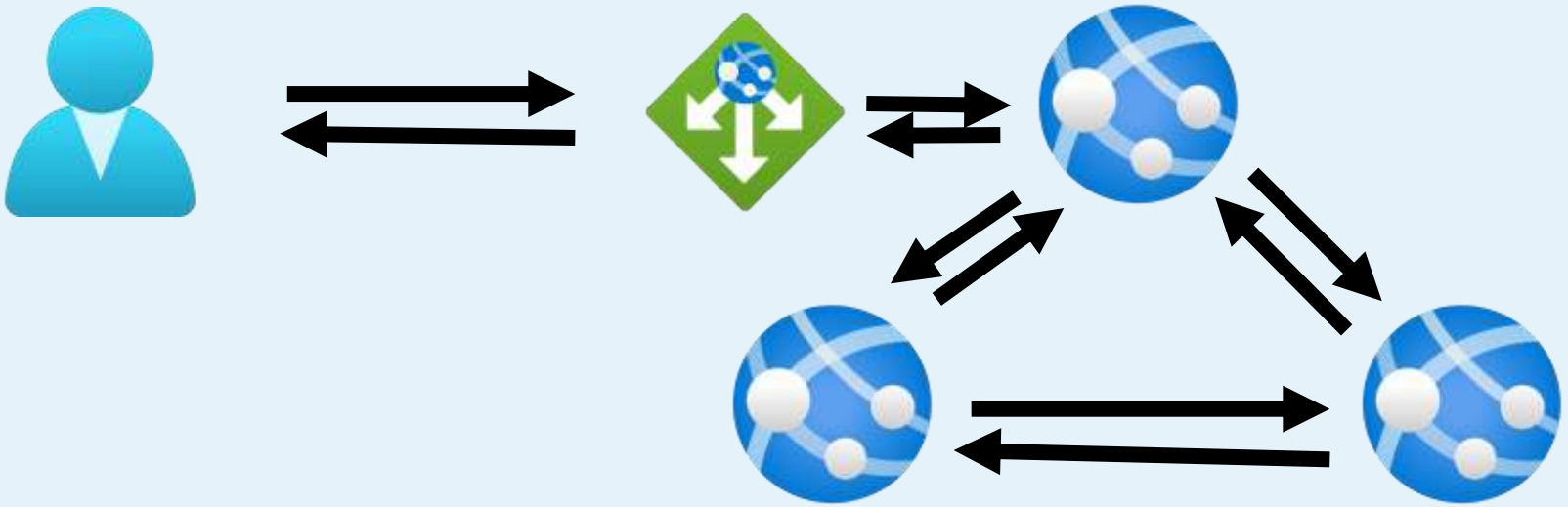
This is the way.



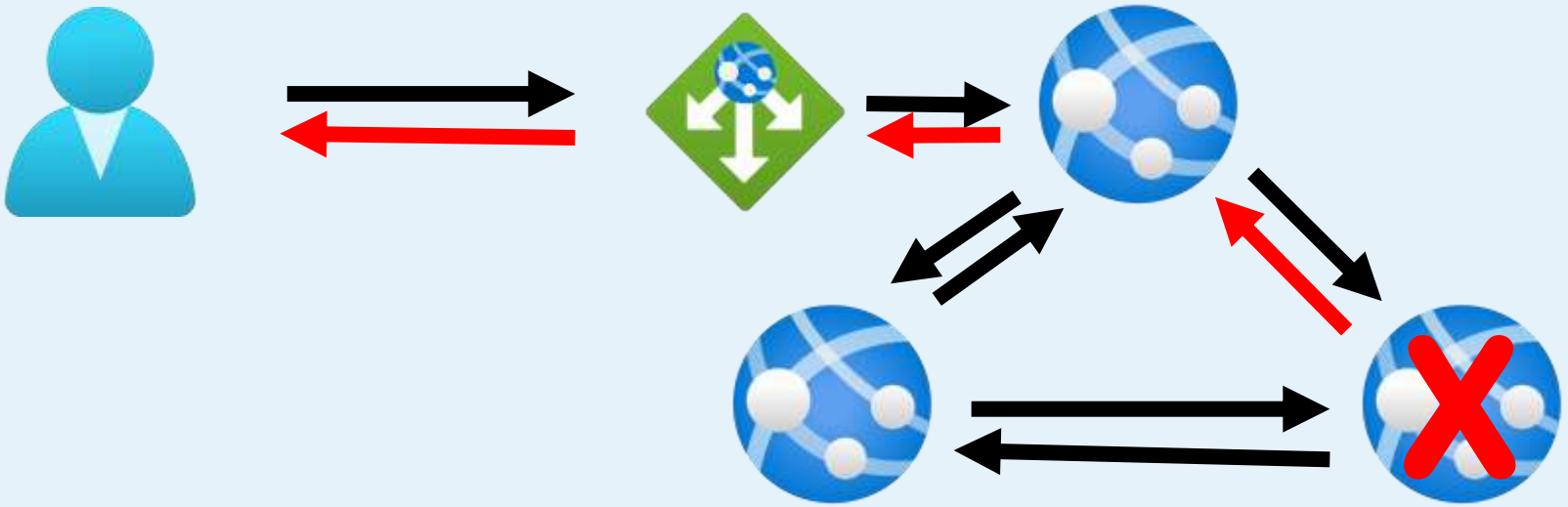
Pitfall 3
Use Synchronous Calls for Everything

HTTP all the things!

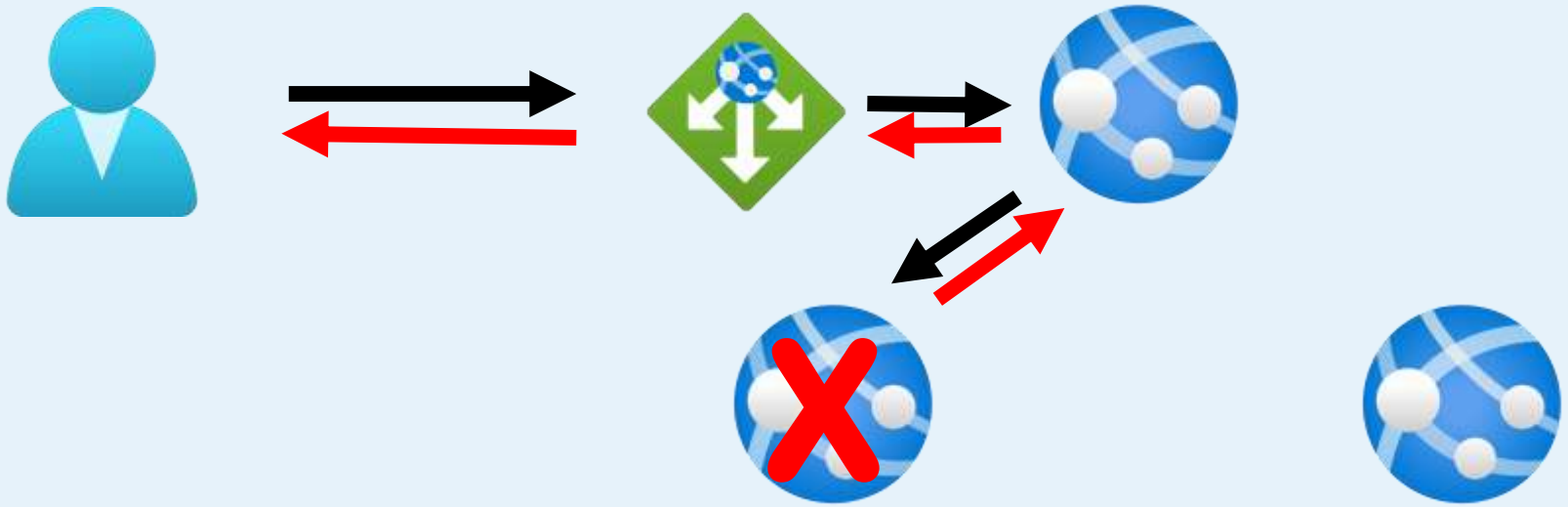
Synchronous HTTP Calls: Performance Impact



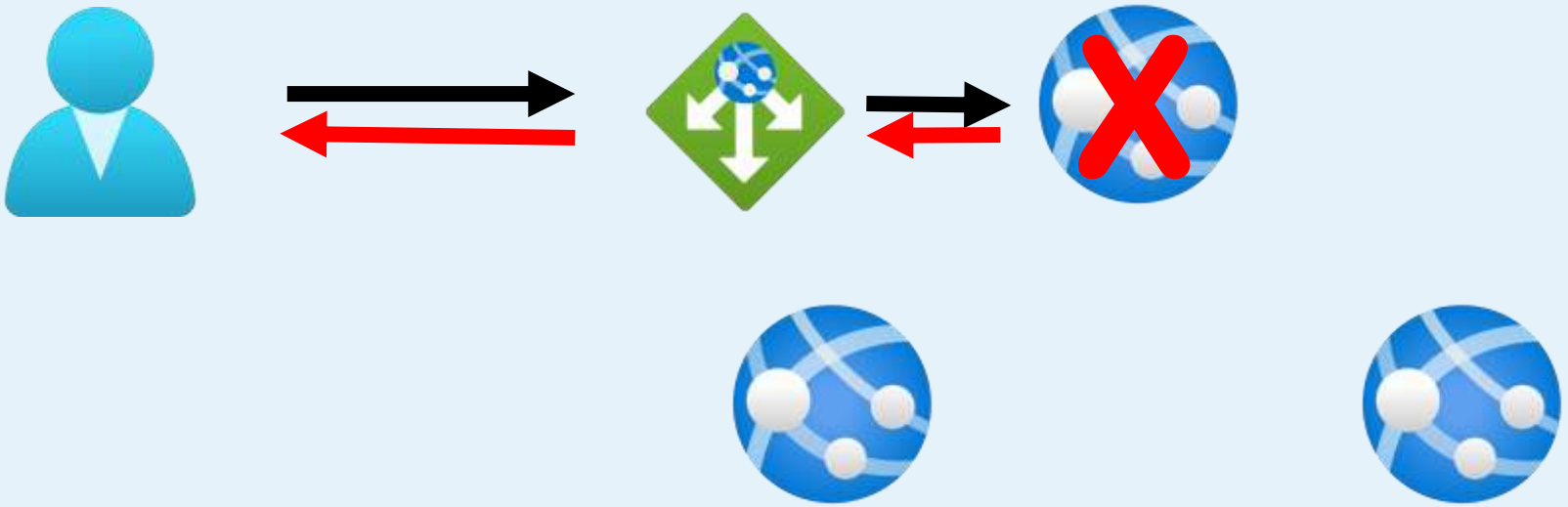
Synchronous HTTP Calls: Availability Impact



Synchronous HTTP Calls: Availability Impact



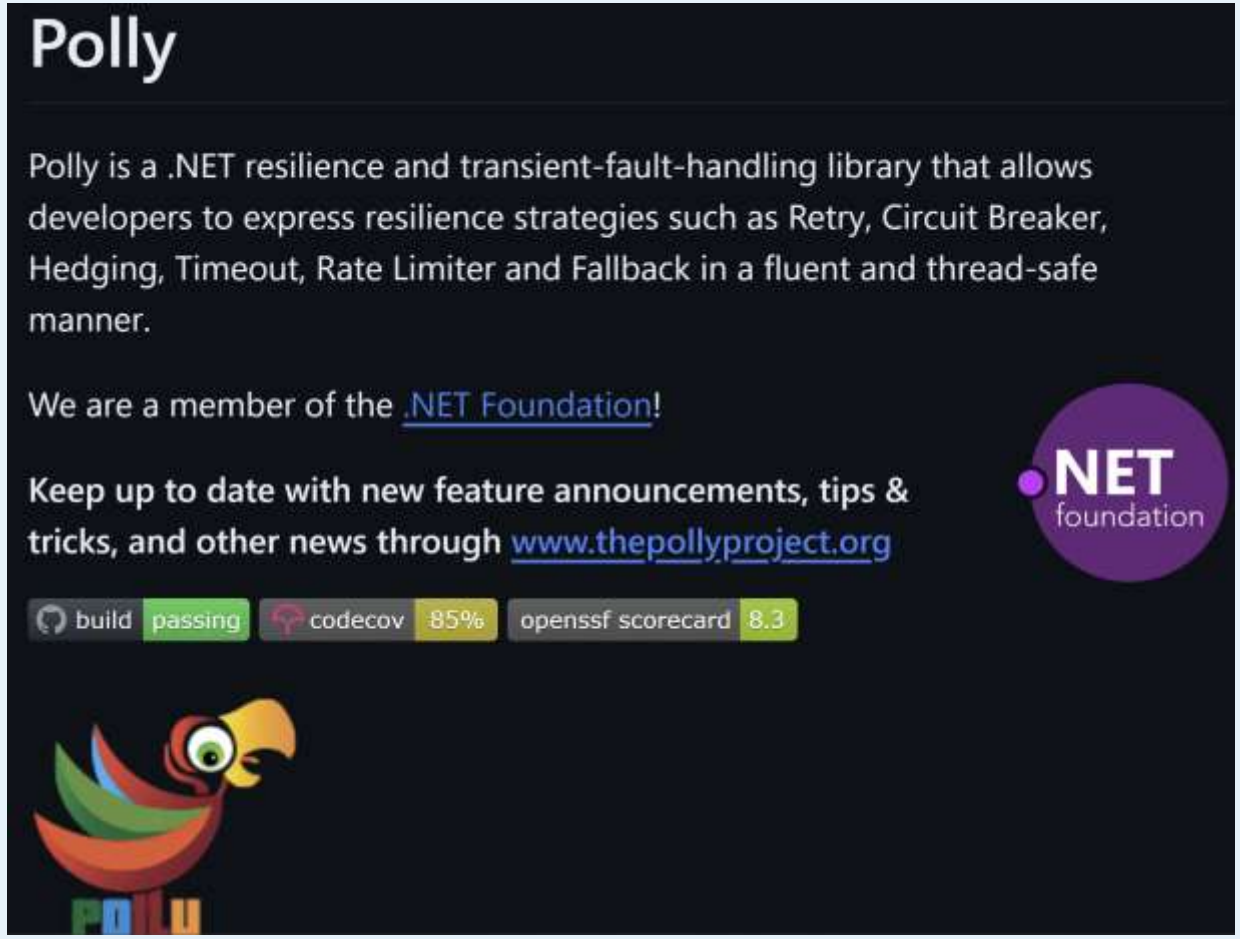
Synchronous HTTP Calls: Availability Impact



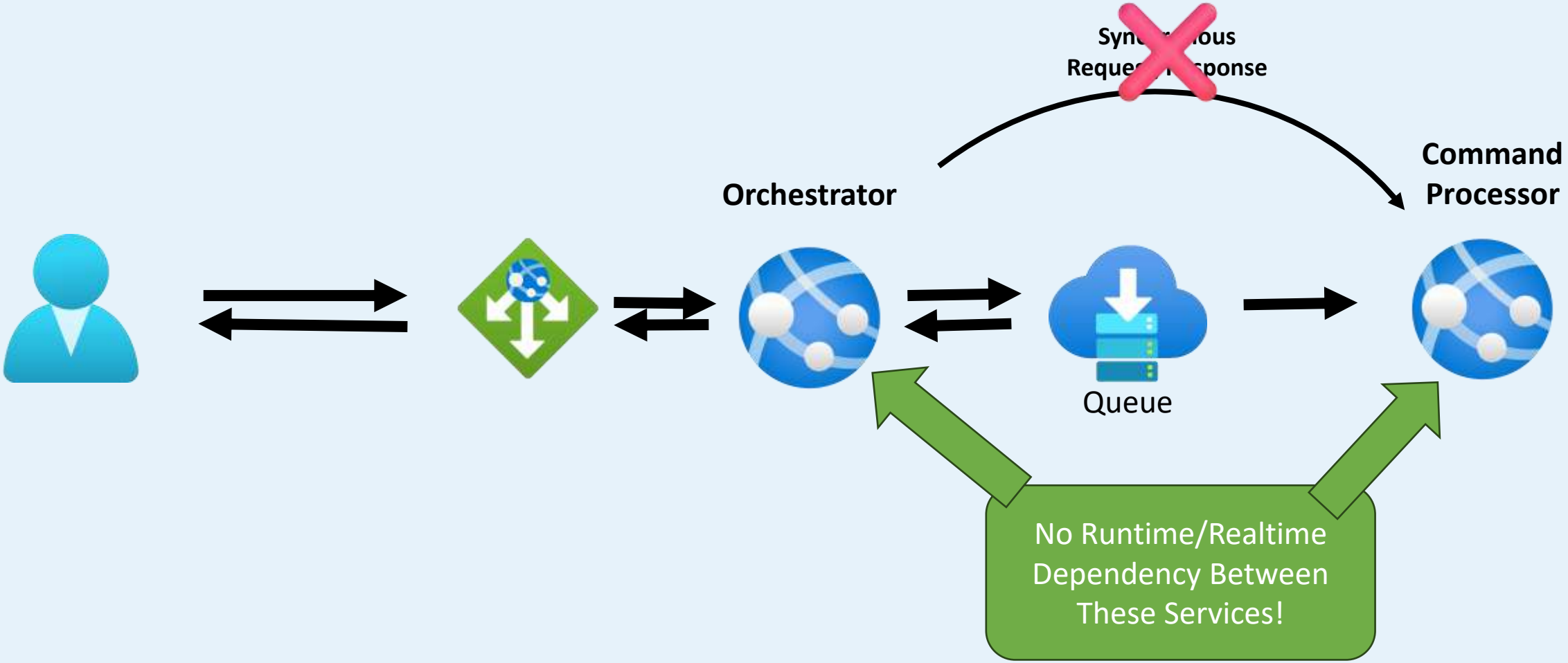
Retries and Circuit Breakers

- When you need synchronous calls
  - Create a **Retry** policy
  - Consider adding a **Circuit Breaker**

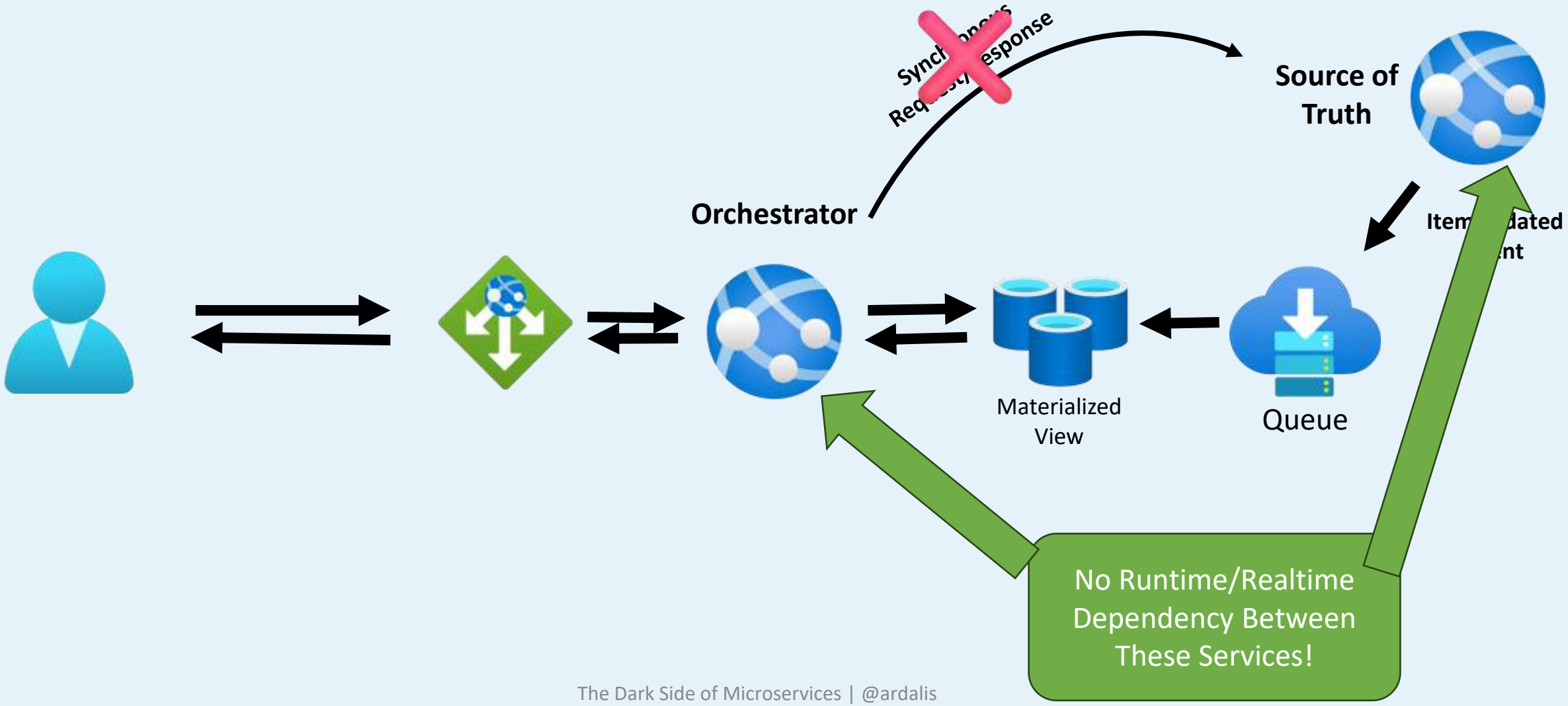
- Consider **Polly** or new .NET 8+ wrappers:
  - Microsoft.Extensions.Resilience
  - Microsoft.Extensions.Http.Resilience



Consider Async Messaging for Commands



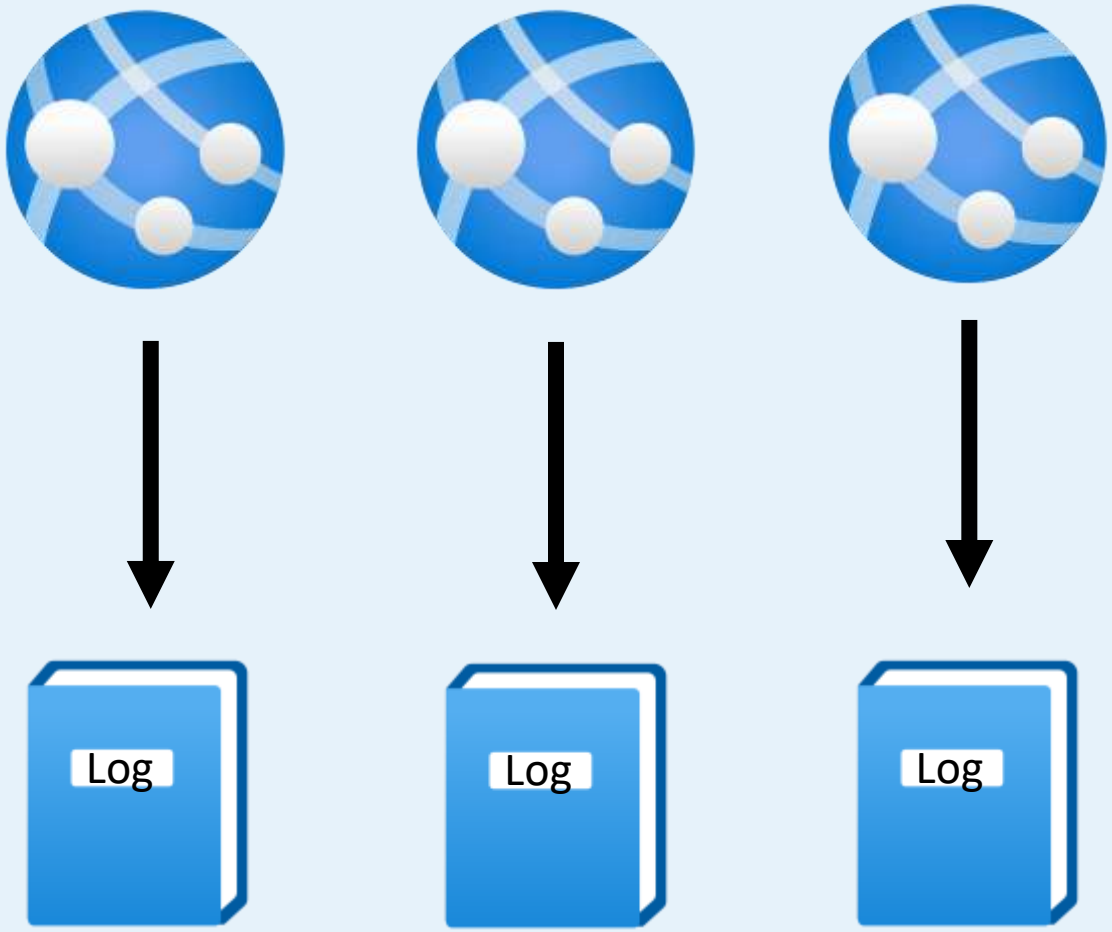
Consider Materialized Views for Queries



Pitfall 4
Logs Per Service

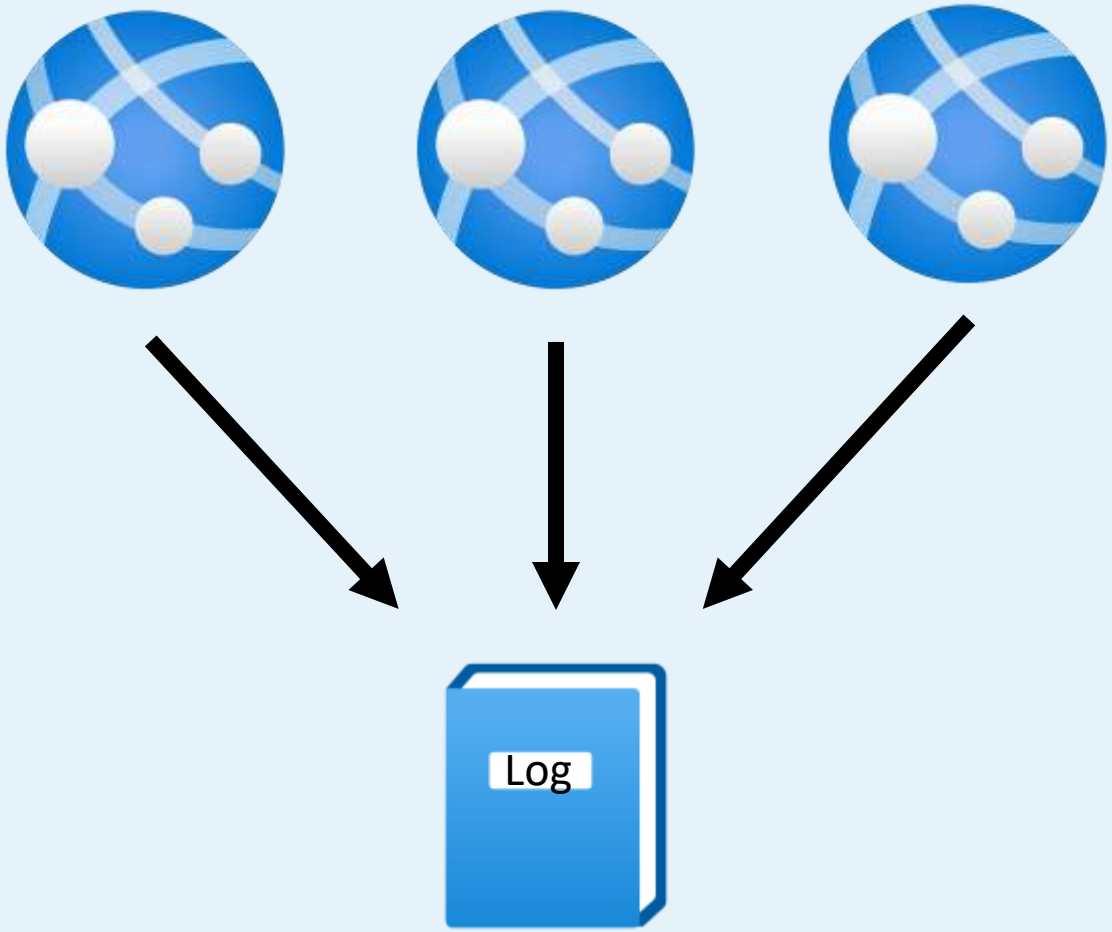
Who needs centralized logging?

This is NOT the way.



Debugging is hard enough without having to hunt down separate requests' logs on separate machines

This is the way.



Structured Logging

AspireSample Dashboard

- Projects
- Containers
- Executables
- Logs
  - Project
  - Container
  - Executable
  - Structured
- Traces
- Metrics

Structured Logs

(All)

Filter... Level: (All) Filters: No Filters

Service	Level	Timestamp	Message	Trace	Details
apiservice	Information	10:43:34.210 AM	Now listening on: https://localhost:7453		View
apiservice	Information	10:43:34.212 AM	Now listening on: http://localhost:5499		View
apiservice	Information	10:43:34.214 AM	Application started. Press Ctrl+C to shut down.		View
apiservice	Information	10:43:34.214 AM	Hosting environment: Development		View
apiservice	Information	10:43:34.214 AM	Content root path: D:\source\repos\docs-aspire\docs\get-started\snippets\quickstart\AspireSample\As...		View
webfrontend	Information	10:43:35.639 AM	Now listening on: https://localhost:7281		View
webfrontend	Information	10:43:35.641 AM	Now listening on: http://localhost:5205		View
webfrontend	Information	10:43:35.643 AM	Application started. Press Ctrl+C to shut down.		View
webfrontend	Information	10:43:35.643 AM	Hosting environment: Development		View
webfrontend	Information	10:43:35.643 AM	Content root path: D:\source\repos\docs-aspire\docs\get-started\snippets\quickstart\AspireSample\As...		View

Total: **10 results found**

Embrace OpenTelemetry (OTEL)

- Traces
- Metrics
- Logs

https://opentelemetry.io/

Let Aspire be your first OTEL consumer

Pitfall 5
Use Picoservices

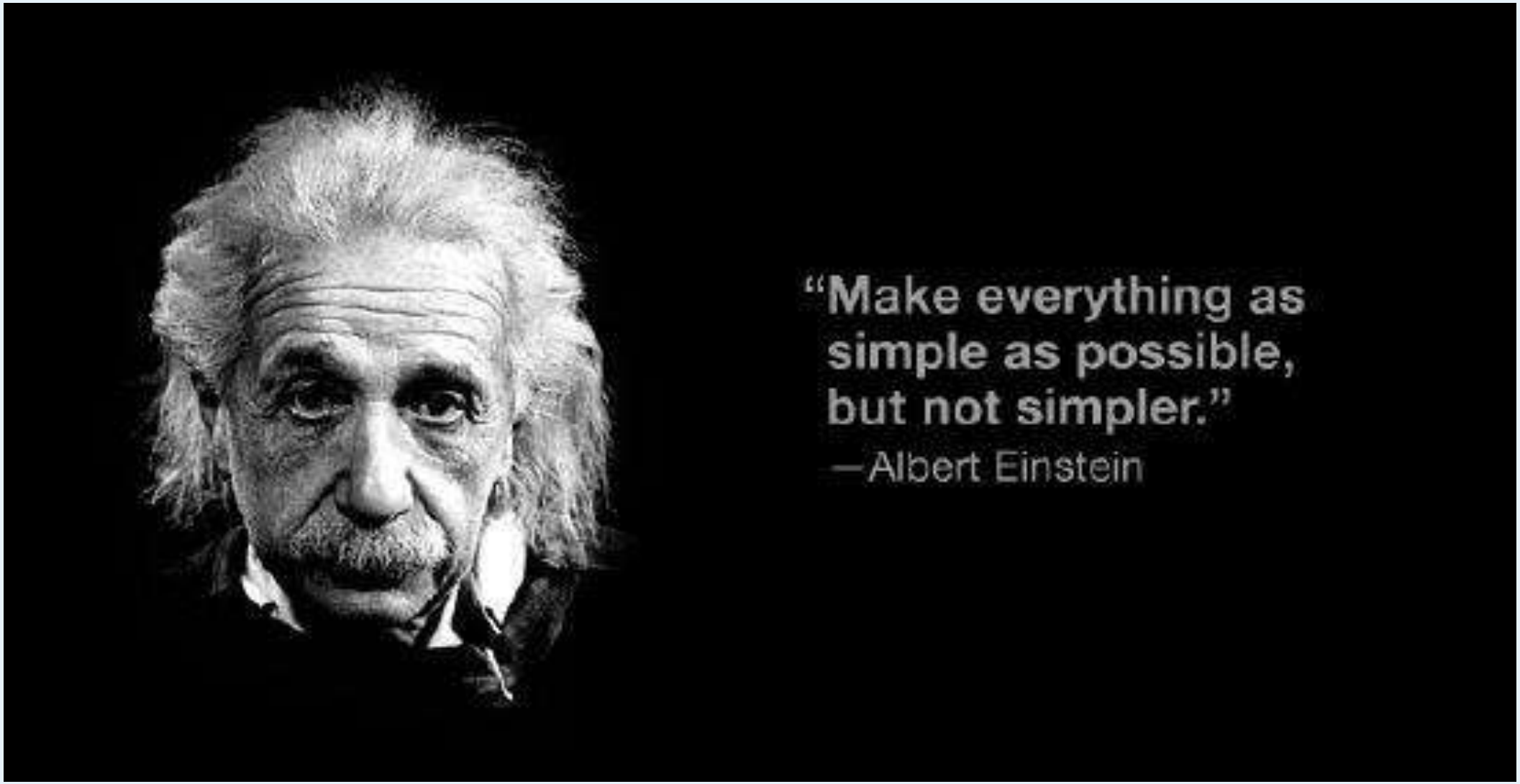
Micro: 10^-6

Nano: 10^-9

Pico: 10^-12 (aka micro-micro-services)

Micro- doesn't (necessarily) mean one function

- Map each service to a **bounded context**
- Services should **enforce encapsulation boundary**
- Services should be large-grained enough to do useful work **without** having to always lean on other services
- When too small, number of services and calls between services both **explode rapidly**


"Make everything as simple as possible, but not simpler."
—Albert Einstein

Strive for “Goldlocks”-sized Services

- Too large, and you lose
  - Independent scalability
  - Easy decomposition into many teams
  - Simplicity within the bounded context
- Too small, and you lose
  - Cohesion
  - Encapsulation
  - In-process performance benefits
  - Lower cost of having fewer services to deploy and maintain

The same holds true for messages

- Send enough data in messages to avoid getting DOS'd by consumers' calls asking for more information
  - Avoid "data deficient messages"

- Weigh this against the need for consumers to have the latest information

Consuming App
Source App
{
"event-type" : "OrderCreated",
"customerId" : "123",
"orderId" : "456"
}
Consuming App
Consuming App

Source App
HTTP GET /customers/123
HTTP GET /orders/456
Consuming App
HTTP GET /customers/123
HTTP GET /orders/456
Consuming App
HTTP GET /customers/123
HTTP GET /orders/456
Consuming App

Pitfall 6
Breaking the Second Rule of Optimization

Don’t do it… **yet**.

Rules of Optimization

1. Don’t.
2. Don’t, yet. (for experts only)
3. Measure first.

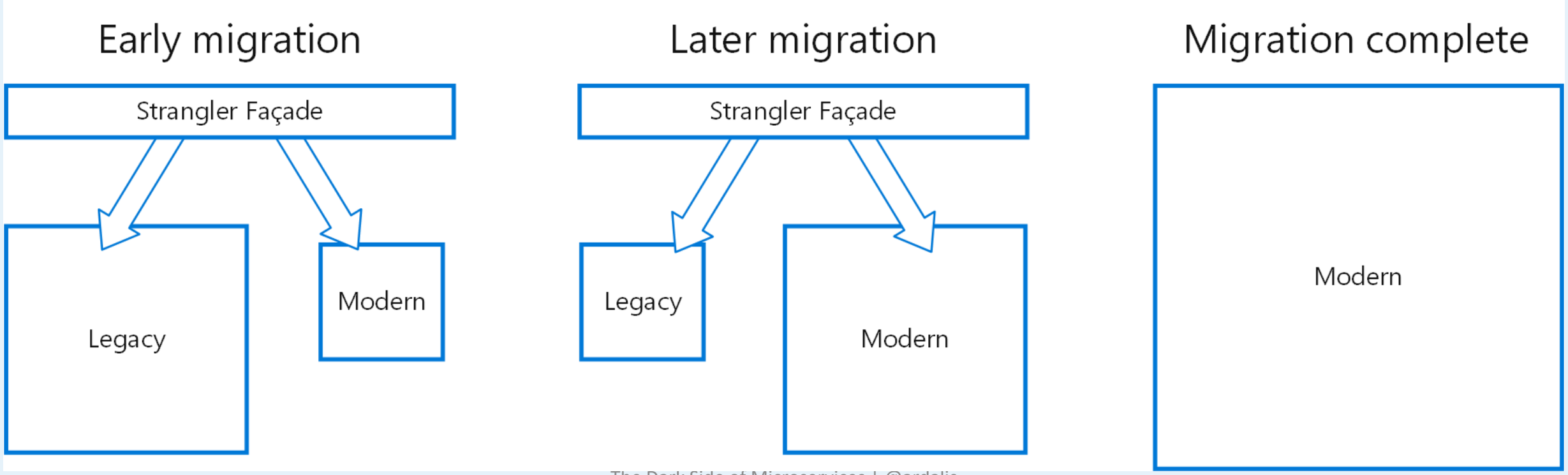
https://wiki.c2.com/?RulesOfOptimization

Microservices **are an optimization**

- Avoid premature optimization
- **First**, build a modular, correct, working, monolithic system
- Identify **actual problems** that a microservices architecture might address
- Spin off **one** microservice and then reassess

Strangler Fig Pattern

- Wrap monolith with a pass-through façade layer
- Gradually migrate functionality to new (microservices) design



https://learn.microsoft.com/en-us/azure/architecture/patterns/strangler-fig

Pitfall 7
Hidden coupling

Microservices should be completely independent of one another.

This is tougher than it sounds.

Service Coupling

- Shared database (covered earlier, **Pitfall 2**)
- Shared DTOs
- Shared helper/utility classes
- Shared messaging schemas
- Temporal coupling
  - “Service A can only be called once a call to Service B has been made.”
- Shared test environments/services
  - Avoid having only one test instance of a service if many other services must integrate with it

Shared DTOs Example

- CustomerDTO
  - ~~Name~~ **Fullname**
  - Addresses
  - PaymentMethods
  - Orders
- OrderFulfillment
  - Update ~~CustomerDTO~~ to change order status
- PaymentService
  - Update ~~CustomerDTO~~ to change status of a payment method to declined
- ShippingService
  - Update ~~CustomerDTO~~ with new shipping address

Requires Redeployment of ALL FOUR SERVICES to Support the New DTO Format

What to use instead? **Be Specific.**

- CustomerOrderUpdateDTO
  - Id
  - Orders
- CustomerPaymentUpdateDTO
  - Id
  - PaymentProviders
- CustomerAddressUpdateDTO
  - Id
  - Addresses

Pitfall 8
Ignoring security

Throw an API Gateway and maybe a WAF in front of your microservices and call it a day.

Securing Microservices

- Yes, you want an API Gateway
- Yes, you want a firewall (Web Application Firewall)

But also:

- TLS everywhere
- Pass through user identity/claims
- Zero trust. Always verify, even for internal requests
- Automated security scans of code and production environment
- Encrypt sensitive data and redact from logs

Split Bounded Contexts in Your Monolith First

- New, Greenfield app:
  - Iterate quickly, optimize rapid learning about the domain and how to model it
  - Refactor the domain and bounded contexts frequently
  - Use **well-designed modules** for each bounded context

- Legacy system:
  - Refactor to pull out individual bounded contexts into **well-designed modules**
  - Retain in-process communication; maximize speed and simplicity of deployments

- Split out individual microservices **after** their boundaries are well understood

https://modularmonoliths.com/

Summary

- **Don't** start with microservices
- **Don't** share databases
- **Don't** overuse synchronous calls between services
- **Don't** keep separate logs per service
- **Don't** make services too small

- **Do** adopt microservices incrementally
- **Do** embrace automation
- **Do** consider versioning early
- **Do** watch out for hidden coupling
- **Do** start with modules before adding network calls

Resources

Find me:

- ardalis.com
- YouTube.com/ardalis
- steve@nimblepros.com
- NimblePros.com

Thanks!